CSE370 : Database Systems

Project Report

Project Title : ICICIDS (Integrated Crime Investigation and Criminal Identification Database System)

Group No : 08, CSE370 Lab Section : 11, Spring 2026		
ID	**Name**	**Contribution**
23201606	MD. Amirul Islam Sadat	Backend and Frontend app Design
23301680	Abraar A Rehman	Frontend Design
24341288	Akaeyd Hossain	Database Design

Table of Contents

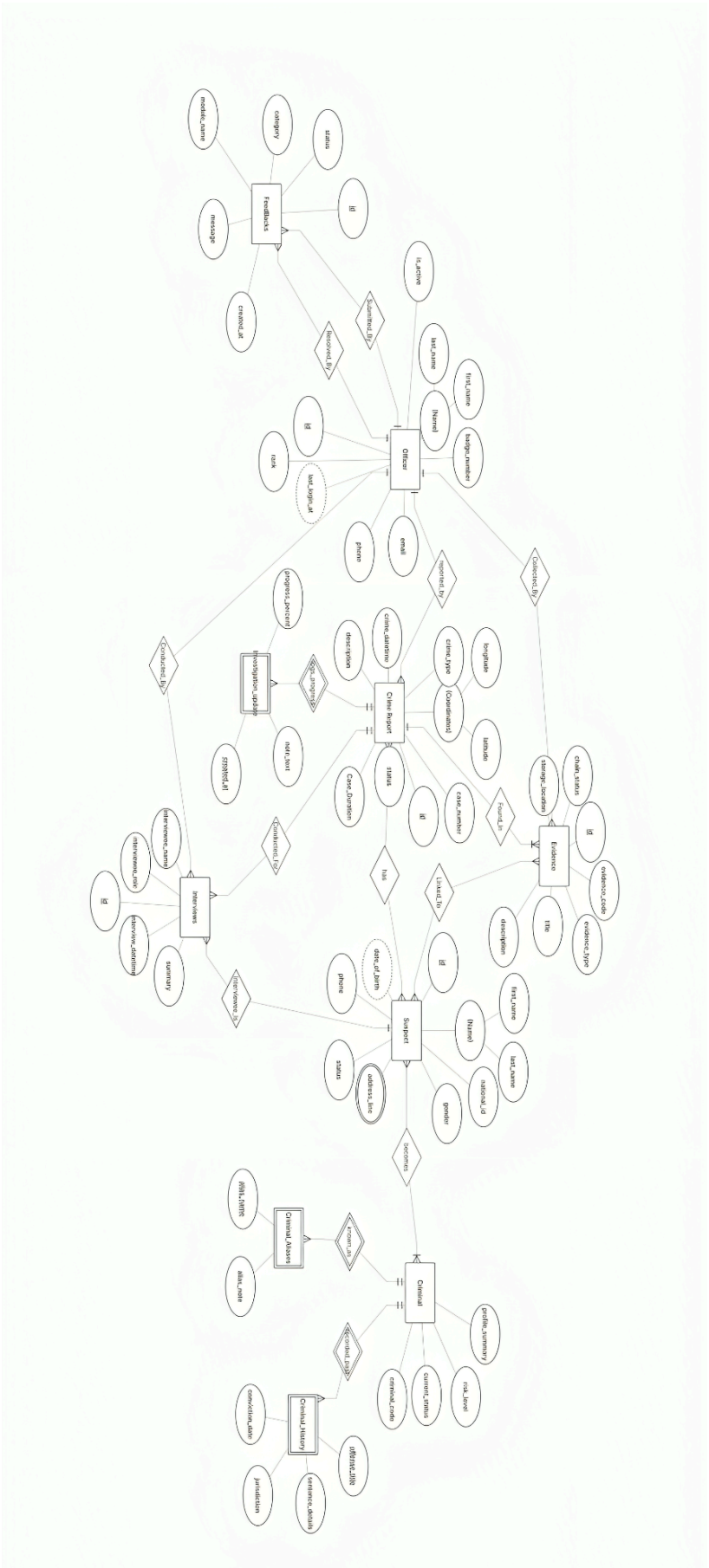
Introduction

The **ICICIDS (Integrated Crime Investigation and Criminal Identification Database System)** is a comprehensive web-based platform designed to streamline crime reporting, suspect tracking, criminal database management, evidence handling, and investigation oversight. The system enables law enforcement officers to report crimes, link suspects and evidence to cases, manage criminal profiles and aliases, track investigation progress through timelines and interviews, and maintain forensic chain-of-custody records. Core features include strict role-based access control (three officer grades with hierarchical permissions), comprehensive audit logging for all activities and login attempts, schema modification requests with approval workflows, and user feedback mechanisms. The platform enforces data integrity through relational database constraints (MySQL/InnoDB), implements server-side and client-side input validation (formatted case numbers, phone numbers, national IDs, criminal codes), and provides a responsive user interface for case management, suspect/criminal record creation, evidence tracking, and investigation timeline visualization. The system prioritizes security through prepared statements (SQL injection prevention), password hashing, geolocation logging, and immutable audit trails, making it suitable for law enforcement agencies requiring certified crime data management and investigative transparency.

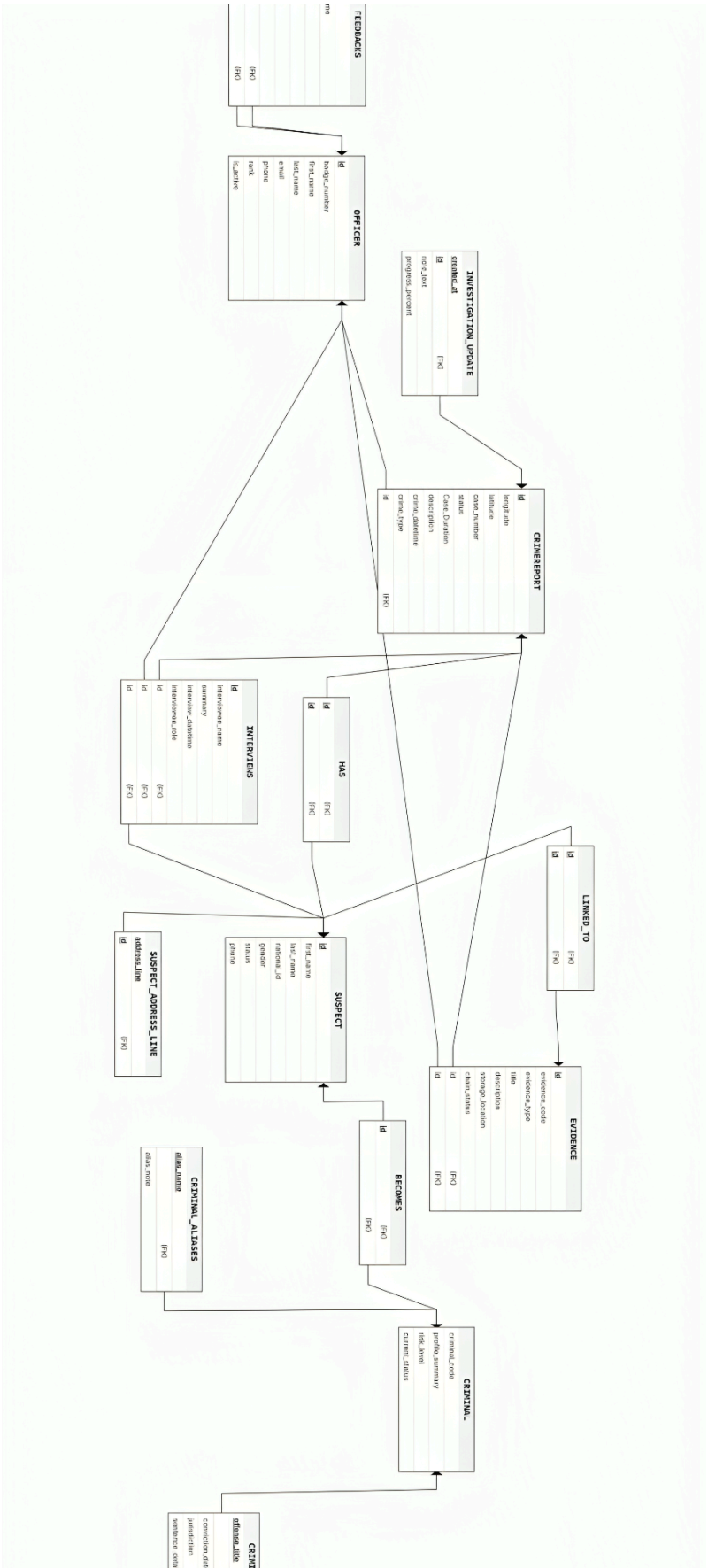
Project Features

ID, Name	Features [3 per member]	
23201606, MD. Amirul Islam Sadat	Ft 1	Role-based access control Crime report management Suspect tracking
	Ft 2	Criminal confirmation and history Evidence management Investigation timeline support
	Ft 3	Audit logging and login history Sign up
23301680, Abraar A Rehman	Ft 1	Schema request workflow
	Ft 2	Structured database integrity
	Ft 3	session handling
24341288, Akaeyd Hossain	Ft 1	Responsive UI
	Ft 2	Client-side input help
	Ft 3	Feedback collection

ER/EER Diagram



Schema Diagram



Normalization

1. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.
2. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?
3. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

a. 1NF Compliance:

The ICICIDS schema is fully compliant with First Normal Form (1NF). All 16 tables contain only atomic (single-valued) attributes with no repeating groups or arrays embedded in cells. ENUM fields store single choices (e.g., rank, investigation_status), TEXT fields store single text blocks, and one-to-many relationships are properly handled through separate junction tables (criminal_aliases, crime_report_suspects, crime_report_evidence) instead of denormalized data. Every table employs proper foreign key constraints to represent relationships, ensuring that each cell contains indivisible, single values. The schema requires no decomposition to achieve 1NF compliance.

b. 2NF Compliance:

The ICICIDS schema is fully compliant with Second Normal Form (2NF), and no partial functional dependencies are present. All tables employ simple (single-column) primary keys, which eliminates partial dependency risk by definition—a non-key attribute cannot have a partial

dependency on a single-column key. Associative/junction tables with composite unique constraints (crime_report_suspects, criminal_aliases, crime_report_evidence) demonstrate proper design with all non-key attributes depending on the *entire* relationship, not just part of it. For example, in crime_report_suspects, attributes such as relation_type and notes describe the link between both the crime and suspect, not individually. The schema requires no decomposition to achieve 2NF compliance.

c. 3NF Compliance:

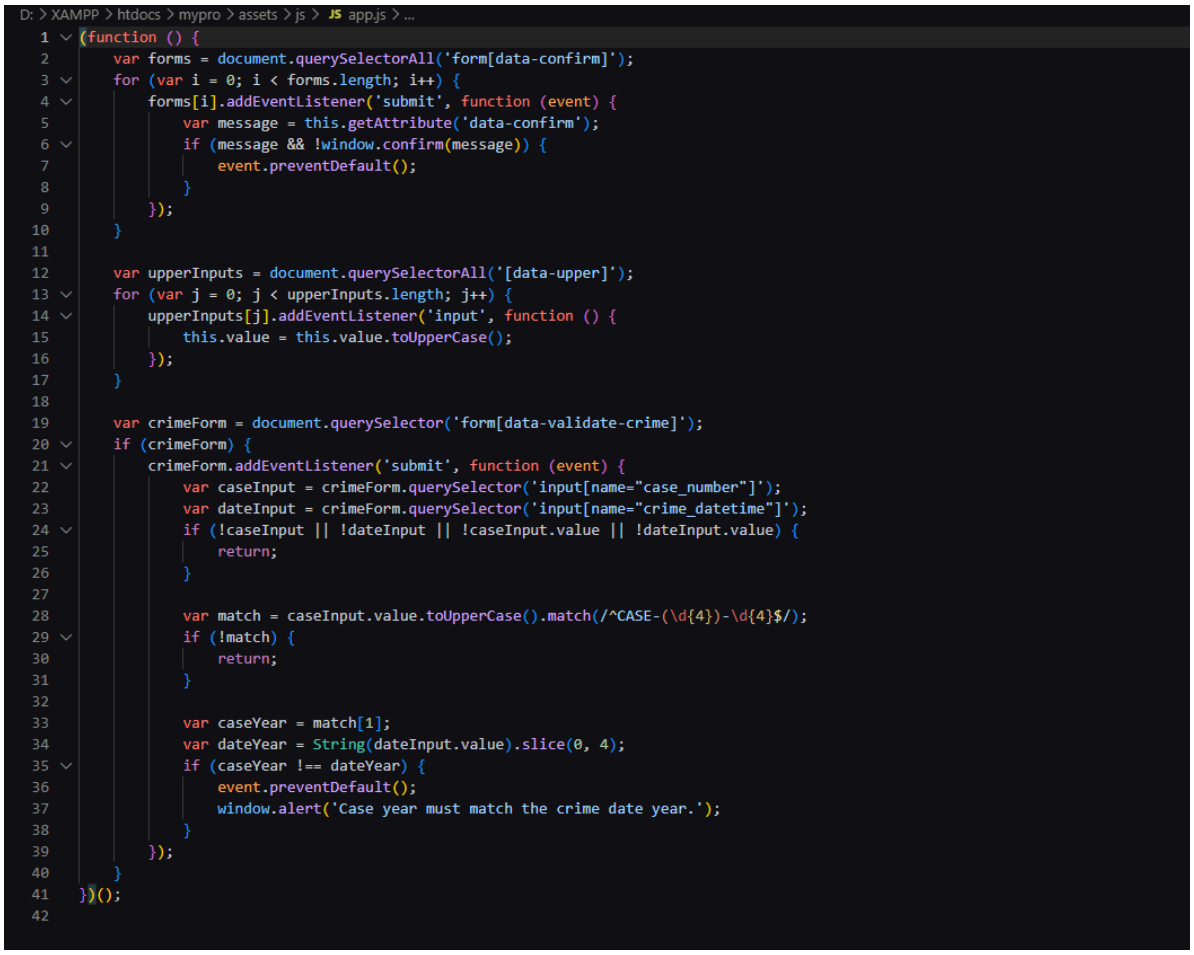
The ICICIDS schema is fully compliant with Third Normal Form (3NF), with only one pragmatic consideration. All non-key attributes in business-logic tables (officers, crime_reports, suspects, criminals, evidence, etc.) depend directly on their primary keys and not on other non-key attributes; no transitive dependencies exist. The only technical exception appears in audit tables (officer_activity and login_logs), where ip_location could theoretically be derived from ip_address. This denormalization is pragmatically justified and represents industry-standard practice for security/audit logging because: (1) geolocation constitutes a third-party lookup service rather than normalized business data; (2) decomposing would introduce significant performance overhead through constant joins; and (3) storing both values is standard forensic practice for completeness. This pragmatic denormalization is acceptable, and the schema requires no further decomposition to achieve 3NF. The database design exhibits production-ready quality with strong data integrity and normalization properties.

Frontend Development

Contribution of ID : 23201606, Name : MD. Amirul Islam Sadat

The ICICIDS frontend JavaScript implementation (assets/js/app.js) provides dynamic client-side functionality and user interaction management. The application handles form validation before submission, including pattern matching for structured inputs such as case numbers (CASE-YYYY-####), national identification (NID-ICI-XXXX), and phone numbers (+880XXXXXXXXX). JavaScript implements auto-uppercase functionality via data-upper attributes on input fields, enforcing consistent data formatting at the point of entry. The application includes cross-field validation logic, specifically, comparing crime date year with case number year to prevent data entry inconsistencies. Event listeners manage form submissions with real-time feedback, form reset operations, modal interactions for delete confirmations, and dynamic role-based UI elements that display or hide sections based on officer permissions received from the server. The JavaScript layer communicates with the PHP backend through standard HTTP POST requests, handling responses and displaying user-friendly error messages when validation fails or referential integrity constraints are violated. This approach provides immediate client-side feedback while maintaining server-side validation as the authoritative

security layer, ensuring that no malformed data reaches the database.



```
D: > XAMPP > htdocs > mypro > assets > js > JS app.js > ...
(function () {
    var forms = document.querySelectorAll('form[data-confirm]');
    for (var i = 0; i < forms.length; i++) {
        forms[i].addEventListener('submit', function (event) {
            var message = this.getAttribute('data-confirm');
            if (message && !window.confirm(message)) {
                event.preventDefault();
            }
        });
    }

    var upperInputs = document.querySelectorAll('[data-upper]');
    for (var j = 0; j < upperInputs.length; j++) {
        upperInputs[j].addEventListener('input', function () {
            this.value = this.value.toUpperCase();
        });
    }

    var crimeForm = document.querySelector('form[data-validate-crime]');
    if (crimeForm) {
        crimeForm.addEventListener('submit', function (event) {
            var caseInput = crimeForm.querySelector('input[name="case_number"]');
            var dateInput = crimeForm.querySelector('input[name="crime_datetime"]');
            if (!caseInput || !dateInput || !caseInput.value || !dateInput.value) {
                return;
            }

            var match = caseInput.value.toUpperCase().match(/^CASE-(\d{4})-\d{4}$/);
            if (!match) {
                return;
            }

            var caseYear = match[1];
            var dateYear = String(dateInput.value).slice(0, 4);
            if (caseYear !== dateYear) {
                event.preventDefault();
                window.alert('Case year must match the crime date year.');
            }
        });
    }
})();

```

Contribution of ID : 23301680, Name : Abraar A Rehman

The ICICIDS frontend styling (assets/css/style.css) implements a responsive, professional interface suitable for law enforcement operational use. The design employs a structured layout with header navigation, role-based menu items, and a main content area organized into logical sections for crime reporting, suspect tracking, criminal database management, and evidence handling. CSS implements responsive grid and flexbox layouts that adapt to various screen sizes, from desktop monitors to tablets, ensuring usability across different field conditions. Color coding and visual hierarchy guide users through complex workflows—form sections are clearly delineated, action buttons are prominent, and status indicators (investigation states,

chain-of-custody stages) use semantic color associations. The styling includes input field styling with clear labels, placeholder text, and visual feedback for required fields and validation states. Tables displaying search results and investigation timelines employ striped rows, hover effects, and clear column headers for readability. Accessibility considerations include sufficient color contrast, readable font sizes, and logical tab ordering to support diverse user needs in operational environments.

```css
:root {
    --bg: #f4f6f9;
    --card: #ffffff;
    --text: #1f2a37;
    --muted: #6b7280;
    --line: #d8dee8;
    --primary: #2b5d8a;
    --primary-dark: #214a6d;
    --ok-bg: #e7f6ea;
    --ok-text: #1f7a3d;
    --err-bg: #fdeaea;
    --err-text: #9b1c1c;
}

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Tahoma, Verdana, sans-serif;
    background: var(--bg);
    color: var(--text);
}

.container {
    max-width: 1100px;
    margin: 0 auto;
    padding: 18px;
}

.card {
    background: var(--card);
    border: 1px solid var(--line);
    border-radius: 8px;
    padding: 14px;
    margin-bottom: 12px;
}

.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 10px;
}
```

```css
h1, h2 {
    margin: 0 0 10px;
}

.help {
    color: var(--muted);
    font-size: 13px;
    margin: 4px 0 0;
}

.tabs {
    display: flex;
    gap: 8px;
    flex-wrap: wrap;
    margin-bottom: 12px;
}

.tabs a {
    text-decoration: none;
    border: 1px solid var(--line);
    color: var(--text);
    background: #fff;
    padding: 8px 12px;
    border-radius: 7px;
    font-size: 14px;
}

.tabs a.active {
    background: var(--primary);
    border-color: var(--primary);
    color: #fff;
}

.stats {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
    gap: 10px;
    margin-bottom: 12px;
}

.stat {
    border: 1px solid var(--line);
    border-radius: 8px;
    background: #fff;
    padding: 10px;
```

```css
.stat span {
    color: var(--muted);
    font-size: 12px;
}

.stat strong {
    display: block;
    margin-top: 6px;
    font-size: 22px;
}

.grid-2 {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 12px;
}

form {
    display: grid;
    gap: 8px;
}

input, select, textarea, button {
    width: 100%;
    border: 1px solid var(--line);
    border-radius: 7px;
    padding: 9px;
    font: inherit;
}

button {
    background: var(--primary);
    color: #fff;
    border: none;
    cursor: pointer;
    font-weight: 700;
}

button:hover {
    background: var(--primary-dark);
}

.btn-secondary {
    background: #4b5563;
}
```

```css
.alert {
    margin-bottom: 10px;
    padding: 9px;
    border-radius: 7px;
    border: 1px solid transparent;
}

.alert.success {
    background: var(--ok-bg);
    color: var(--ok-text);
    border-color: #bde0c5;
}

.alert.error {
    background: var(--err-bg);
    color: var(--err-text);
    border-color: #f1c1c1;
}

.table-wrap {
    overflow-x: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
    font-size: 14px;
}

th, td {
    border-bottom: 1px solid #e8edf4;
    padding: 8px;
    text-align: left;
}

th {
    background: #f0f4fa;
}

.login-box {
    max-width: 420px;
    margin: 60px auto;
}

.logo {
    width: 90px;
    height: 90px;
```

```
191
192  .logo-small {
193      width: 52px;
194      height: 52px;
195      margin: 0 0 8px;
196  }
197
198  .hero-banner {
199      width: 100%;
200      max-height: 220px;
201      object-fit: cover;
202      border-radius: 8px;
203      border: 1px solid var(--line);
204  }
205
206  @media (max-width: 860px) {
207      .grid-2 {
208          grid-template-columns: 1fr;
209      }
210  }
211
```

Backend Development

Contribution of ID : 23201606, Name : MD. Amirul Islam Sadat

Architecture and Core Components

The ICICIDS backend is implemented in PHP 8+ with strict type declarations, employing a service-oriented architecture that separates concerns across multiple layers. The entry point (index.php) functions as the central router and controller, handling all HTTP requests (GET for view rendering, POST for action processing) and orchestrating interactions between the Database singleton, Authentication service, Audit logging service, and domain-specific managers (CrimeAndSuspectManager, EvidenceAndInvestigation). The Database.php class provides PDO connection management with prepared statements as the standard for all SQL execution, preventing SQL injection vulnerabilities while ensuring consistent database abstraction. This architecture enables clean separation between request handling, business logic, and data persistence, facilitating maintainability and testability. Authentication and Role-Based Access Control (RBAC) The Auth.php class implements comprehensive authentication and authorization logic using PHP sessions and hierarchical role definitions (GRADE_1, GRADE_2, GRADE_3 officers). Authentication methods enforce strict permission checks before any operation, with the enforce() method raising exceptions if an officer lacks required permissions for a specific action on a table. Password hashing utilizes PHP's password_hash() with PASSWORD_DEFAULT algorithm, providing secure credential storage resistant to brute-force and rainbow table attacks. Login operations capture comprehensive audit metadata including user agent, IP address, device fingerprint, and optional geolocation, enabling forensic tracking of access patterns. The auth object is passed through all service classes, ensuring that every

database operation validates the current officer's permissions before execution. Business Logic and Data Integrity Domain managers (CrimeAndSuspectManager and EvidenceAndInvestigation) encapsulate business logic for crime reporting, suspect management, criminal database operations, and evidence tracking. These services implement strict input validation at the application layer, case numbers must match CASE-YYYY-#### format with year validation, phone numbers must begin with +880, national IDs follow NID-ICI-XXXX pattern, and criminal codes normalize to CRIM-#### prefixed format. Database transactions ensure atomic operations for complex workflows (linking suspects to crimes, confirming criminals from suspects), preventing partial state updates. The AuditLogger.php service records all modifications with complete context, officer ID, action type, affected table, record ID, timestamp, and operational details, creating immutable audit trails for compliance and forensic investigation. Foreign key constraints at the database level enforce referential integrity, with cascading deletes for dependent records and restrict policies for critical relationships, ensuring data consistency even during concurrent operations.

Source Code Repository

https://github.com/amirulislamsadat-collab/CSE370-Project--ICICIDS

Conclusion

The ICICIDS (Integrated Crime Investigation and Criminal Identification Database System) represents a comprehensive, production-ready solution for law enforcement crime reporting and criminal identification management. The system architecture demonstrates rigorous adherence to database normalization principles, achieving full compliance with Third Normal Form (3NF) across all 16 relational tables, thereby ensuring data integrity, eliminating redundancy, and minimizing maintenance anomalies. The implementation incorporates strict role-based access control (RBAC) with hierarchical officer grades, comprehensive audit logging mechanisms, and immutable forensic trails that provide accountability and transparency in investigative operations. Security measures including prepared parameterized queries, password hashing, geolocation tracking, and multi-layered input validation (server-side and client-side) effectively mitigate common web application vulnerabilities such as SQL injection and unauthorized access. The schema design facilitates complex investigative workflows through properly designed associative entities (crime_report_suspects, crime_report_evidence, suspect_evidence), enabling officers to link suspects, evidence, and criminal histories to cases while maintaining referential integrity through cascade and restrict delete constraints. The platform's responsive user interface, combined with server-side business logic implemented in PHP with PDO database abstraction, provides a maintainable, scalable foundation for law enforcement agencies requiring certified criminal investigation and identification systems. This project successfully demonstrates the

integration of database theory, security principles, software engineering practices, and practical law enforcement requirements into a coherent, deployable information management system suitable for institutional use in criminal investigation and offender identification workflows.

References

Codd, E. F. (1970). A relational model of data for large shared data banks. ***Communications of the ACM*****, 13(6), 377–387. https://doi.org/10.1145/362384.362685**

ISO/IEC 27001:2022. (2022). ***Information security, cybersecurity and privacy protection—Information security management systems, Requirements*****. International Organization for Standardization.**

Kent, W. (1983). A simple guide to five normal forms in relational database theory. ***Communications of the ACM*****, 26(2), 120–125. https://doi.org/10.1145/358141.358153**

OWASP. (2021). ***OWASP top 10 – 2021*****. The Open Worldwide Application Security Project. https://owasp.org/Top10/**

Sandhu, R. S., Coynek, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. ***IEEE Computer*****, 29(2), 38–47. https://doi.org/10.1109/2.485845**